XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

EEE312 Week 9
Lab Practice

Use the Newton-Raphson method to write a C program to find the roots of the following nonlinear equation:

$$f(x) = 3x^4 - 5x^2 + 3x + 1 \ , x \in [-1, 3]$$

Start the iteration from initial value x0= 0.

Observe how the solution converges by displaying the results at every iteration of the algorithm.

The stopping condition is until $| x_{i+1} - x_i | < \varepsilon$, where $\varepsilon = 0.000001$.

// Beginning of the C Source code

```
1   // Newton-Raphson
2   #include <stdio.h>
3   #include <math.h>
4
5   float nonlinearFunction(float x)
6   {
7       float y;
8
9       y = 3.0*pow(x,4)-5.0*pow(x,2)+3.0*x + 1;
10
11      return y;
12  }
13
14  float derivativeFunction(float x)
15  {
16      float y;
17
18      y = 12.0*pow(x,3) - 10.0*x + 3.0;
19
20      return y;
21  }
```

Line 5-11: Define the function for nonlinear function $f(x)$:

$$f(x) = 3x^4 - 5x^2 + 3x + 1$$

Line 14-20: Define the function for the derivative of the nonlinear function *df*(*x*)/*dx*:

$$f'(x) = 12x^3 - 10x + 3$$

Function main()

```
23  int main()
24  {
25      float x = 0.0; // Initialize x for the iteration i = 0
26      float new_x = 0.0; // store the new iteration value x
27      float f_x = 0.0; // store the function evaluated at x
28      float dfx = 0.0; // store the df/dx value at x
29      float error = 0.0; // store the error=fabs(x-x_new)
30      float epsilon = 0.000001; // stop the iteration if
31                        // error < epsilon
```

Line 25: Initialize x= 0.0 for the first iteration i= 0.

Line 26: Declare a variable new_xto store the new value xat next iteration i+1.

Line 27: Declare a variable f_xto store the value of the function f(x).

Line 28: Declare a variable dfxto store the value of df(x)/dx.

Line 29: Declare a variable to store the error= $| x_i - x_{i+1} | < \varepsilon$.

Line 30: Declare a variable to store ε = 0.000001.

```
33      do {
34          f_x = nonlinearFunction(x);
35          dfx = derivativeFunction(x);
36          new_x = x - f_x/dfx; // update new value of x
37          printf("\nx=%f", new_x);
38          error = fabs(x-new_x); // calculate the error
39          printf("\nError=%f", error);
40          x = new_x; // store the x_new in current x
41      }while(error>epsilon);
42      printf("\nThe root of the nonlinear equation is x = %f.\n", new_x);
43      return 0;
44  }
```

Line 34: Evaluate f(x) at x value.

Line 35: Evaluate df(x)/dx at x value.

Line 36: Update the new value of x using the Newton-Raphson equation

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

```
33      do {
34          f_x = nonlinearFunction(x);
35          dfx = derivativeFunction(x);
36          new_x = x - f_x/dfx; // update new value of x
37          printf("\nx=%f", new_x);
38          error = fabs(x-new_x); // calculate the error
39          printf("\nError=%f", error);
40          x = new_x; // store the x_new in current x
41      }while(error>epsilon);
42      printf("\nThe root of the nonlinear equation is x = %f.\n", new_x);
43      return 0;
44  }
```

Line 37: Print out the new value of x.

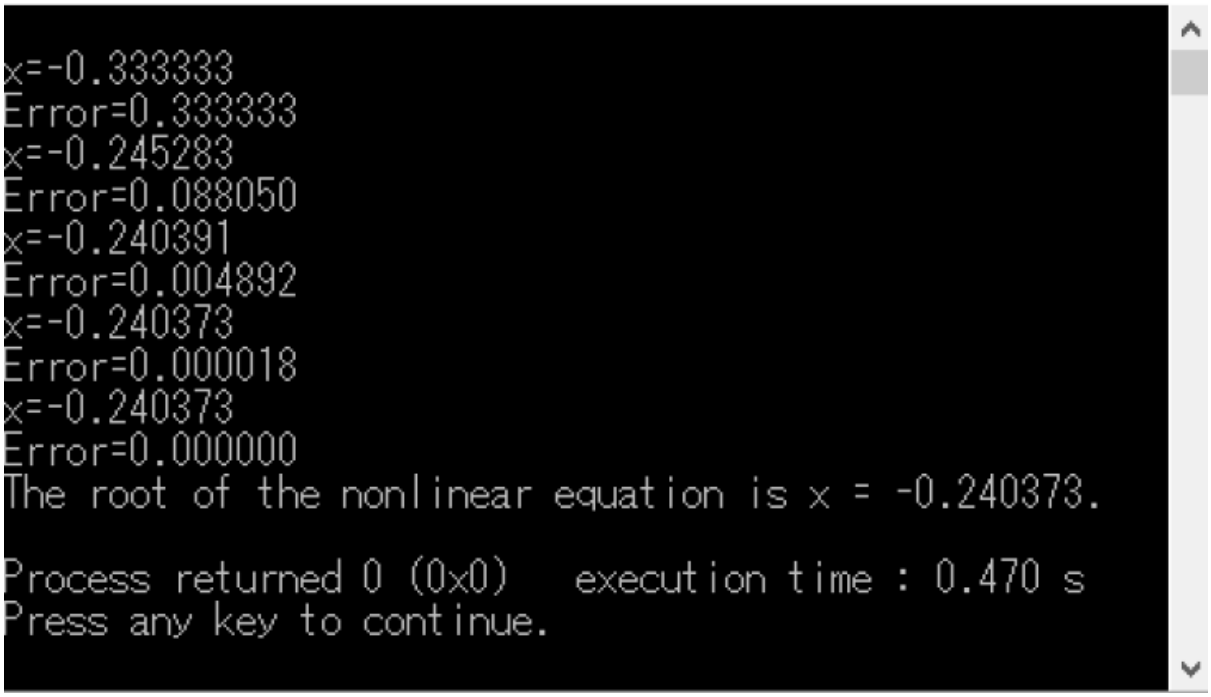
Line 38: Calculate the error using the formula error = | $x_i - x_{i+1}$ |

Line 39: Print out the error value.

Line 40: Store the updated x_newin the current xfor the next iteration.

Line 41: Check if error > epsilon. If yes, return to the beginning of the do-while loop and repeat the algorithm again. Otherwise, escape from the loop and output the final solution in Line 42.

Output


x=-0.333333
Error=0.333333
x=-0.245283
Error=0.088050
x=-0.240391
Error=0.004892
x=-0.240373
Error=0.000018
x=-0.240373
Error=0.000000
The root of the nonlinear equation is x = -0.240373.
Process returned 0 (0x0) execution time : 0.470 s
Press any key to continue.